Generative AI

Handbook

A comprehensive guide covering generative models, LLM architectures, Transformers, GPT, prompt engineering, diffusion models, multimodal AI, RAG, and MCP.

By: Hrithik Gavankar

Open-source educational resource

Table of Contents

1. **Overview & Topics Covered**
2. **Module 1: Foundations of Generative AI**
    - 2.1 Introduction to Generative AI
    - 2.2 How Generative AI Works
    - 2.3 Traditional NLP & Language Models
    - 2.4 Word Representations
    - 2.5 Neural Sequence Models: RNN & LSTM
    - 2.6 Attention Mechanism
    - 2.7 Transformers
3. **Module 2: GPT, LLMs & Applications**
    - 3.1 Large Language Models (LLMs)
    - 3.2 GPT: Architecture & Training
    - 3.3 Prompt Engineering
    - 3.4 Advanced Prompt Engineering
    - 3.5 Image/Video Generation & Diffusion Models
    - 3.6 Multimodal AI
    - 3.7 Ethical Considerations in Generative AI
    - 3.8 Generative AI Tools & Applications
    - 3.9 RAG (Retrieval-Augmented Generation)
    - 3.10 MCP (Model Context Protocol)
4. **Key Formulas & Diagrams Reference**
5. **Practice Questions**
6. **References & Recommended Reading**

Chapter 1: Overview & Topics Covered

About This Handbook

This handbook provides a comprehensive overview of Generative AI. It covers the evolution of language models, LLM architectures, the Transformer revolution, GPT and its training pipeline, prompt engineering techniques, diffusion models, multimodal AI, and practical engineering topics like RAG and MCP. It is designed as a self-contained reference for learners and practitioners.

Learning Objectives

- Understand the core concepts and history of Generative AI
- Learn how different generative models work (VAEs, GANs, Autoregressive, Diffusion)
- Understand ethical considerations and challenges in AI
- Master LLM architectures: from RNNs to Transformers
- Learn GPT architecture, training, and its significance
- Apply prompt engineering techniques effectively
- Understand image/video generation and multimodal AI
- Build practical systems using RAG and MCP

Topics at a Glance

Module 1: Foundations

- Introduction to Generative AI: Definition, scope, overview of generative models and their applications
- Importance of Generative AI in various domains; ethical considerations and challenges
- Introduction to language models and their role in AI; traditional approaches to language modelling
- Overview of popular LLM architectures: RNNs, LSTMs, and Transformers

Module 2: GPT, LLMs & Applications

- Introduction to GPT and its significance; pre-training and fine-tuning processes; architecture and working of GPT models
- Overview of GPT variants and their use cases
- ChatGPT: A practical application of GPT; introduction to ChatGPT and its purpose
- Ethical considerations in Generative AI; understanding ethical implications of generative models
- Prompt engineering: basic and advanced techniques

- Image/video generation, diffusion models, and multimodal AI
- Generative AI tools across domains: Storytelling, Text-to-Image, Chatbots, Text-to-Music, Language Translation, Travel Itinerary, Presentation Tools, Team Building, Education
- RAG (Retrieval-Augmented Generation) and MCP (Model Context Protocol)

Chapter 2: Module 1 - Foundations of Generative AI

2.1 Introduction to Generative AI

What is Generative AI?

Generative AI refers to a class of artificial intelligence systems that can create new content - including text, images, audio, video, and code - by learning patterns from existing data. Unlike discriminative models that classify or predict labels, generative models learn the underlying distribution of data and can produce novel outputs that resemble the training data.

Definition and Scope

Generative AI encompasses any AI technique capable of producing new, previously unseen content. The scope includes:

- Text Generation: Writing articles, stories, code, and conversational responses
- Image Generation: Creating realistic images, artwork, and designs from text descriptions
- Audio Generation: Producing music, speech synthesis, and sound effects
- Video Generation: Creating and editing video content
- Code Generation: Writing, completing, and debugging software code
- Multimodal Generation: Combining multiple modalities (text + image, text + audio, etc.)

Types of Generative Models

The major categories of generative models include:

1. Variational Autoencoders (VAEs)

VAEs learn a compressed latent representation of data. They consist of an encoder that maps input to a latent space and a decoder that reconstructs data from the latent representation. They introduce randomness through sampling, enabling generation of new data.

2. Generative Adversarial Networks (GANs)

GANs use two competing neural networks: a Generator that creates fake data and a Discriminator that tries to distinguish real from fake. Through this adversarial training, the generator learns to produce increasingly realistic outputs.

3. Autoregressive Models

These models generate data one element at a time, where each new element is conditioned on previously generated elements. GPT is the most prominent example, generating text token by token.

4. Diffusion Models

These models learn to generate data by reversing a gradual noising process. They start with pure noise and iteratively denoise it to produce high-quality outputs. Used in DALL-E, Stable Diffusion, and Midjourney for image generation.

5. Transformer-based Models

Built on the Transformer architecture (self-attention mechanism), these models have become the foundation of modern Generative AI. Examples include GPT, BERT, T5, and LLaMA.

Applications of Generative AI

- Healthcare: Drug discovery, medical image synthesis, clinical note generation
- Entertainment: Game design, music composition, movie special effects
- Education: Personalized tutoring, content creation, automated assessment
- Business: Marketing content, customer service chatbots, report generation
- Software Development: Code generation, bug fixing, documentation
- Science: Protein structure prediction, materials design, simulation
- Creative Arts: Digital art, creative writing, design prototyping

2.2 How Generative AI Works

Understanding how Generative AI works requires grasping three fundamental concepts: tokens, context windows, and temperature. These form the operational backbone of modern generative models.

Tokens

Tokens are the fundamental units that language models process. A token can be a word, a part of a word (subword), or even a single character. Modern models use subword tokenization algorithms like Byte-Pair Encoding (BPE).

- A token is not always a complete word. For example, 'unbelievable' might be split into 'un', 'believ', 'able'
- On average, 1 token is approximately 3/4 of a word in English
- Tokenization allows models to handle rare words, neologisms, and multiple languages
- The vocabulary size (number of unique tokens) affects model capacity and efficiency
- GPT-4 uses roughly 100,000 tokens in its vocabulary

Context Window

The context window is the maximum number of tokens a model can process in a single interaction. It includes both the input (prompt) and the output (response).

- GPT-3.5: ~4,096 tokens; GPT-4: up to 128,000 tokens
- A larger context window allows the model to consider more information when generating responses
- The context window determines how much 'memory' the model has during a conversation
- Exceeding the context window causes the model to lose earlier information

Temperature

Temperature is a parameter that controls the randomness/creativity of the model's output. It adjusts the probability distribution over the next token.

- Temperature = 0: Deterministic output; model always picks the highest-probability token (greedy decoding)
- Temperature ~0.3-0.7: Balanced between creativity and coherence
- Temperature = 1.0: Standard sampling from the learned distribution
- Temperature > 1.0: More random, creative, but potentially less coherent outputs
- Low temperature is ideal for factual tasks; high temperature for creative tasks

Key Insight: Temperature does not change the model - it changes how the model samples from its predictions.

2.3 Traditional NLP & Language Models

What is NLP?

Natural Language Processing (NLP) is a branch of AI concerned with enabling computers to understand, interpret, and generate human language. Before modern deep learning, NLP relied heavily on rule-based systems and statistical methods.

What is a Language Model?

A language model is a mathematical model that assigns probabilities to sequences of words. Given a sequence of words, a language model predicts the probability of the next word. Formally: P(w_t | w_1, w_2, ..., w_{t-1}) - the probability of word w_t given all preceding words.

N-Gram Models

N-grams are one of the earliest statistical approaches to language modelling. An N-gram model estimates the probability of a word based on the previous (N-1) words.

- Unigram (N=1): Treats each word independently - P(w)
- Bigram (N=2): Probability based on previous word - P(w_t | w_{t-1})
- Trigram (N=3): Probability based on two previous words - P(w_t | w_{t-2}, w_{t-1})
- Limitation: Cannot capture long-range dependencies (e.g., subject-verb agreement across many words)
- Limitation: Suffers from data sparsity for higher-order N-grams
- Markov Assumption: Each word depends only on the previous N-1 words

2.4 Word Representations

Computers cannot process raw text directly. Words must be converted into numerical representations. The evolution of word representations is a key story in NLP.

A. Bag of Words (BoW)

Bag of Words represents text as a vector of word counts, ignoring grammar and word order.

- Creates a vocabulary from all unique words in the corpus
- Each document is represented as a vector of word frequencies
- Advantage: Simple and easy to implement
- Limitation: Loses word order - 'dog bites man' and 'man bites dog' have the same representation
- Limitation: Cannot capture semantic similarity between words

Example: Consider two sentences: 'The cat sat on the mat' and 'The dog sat on the log'. BoW would create vectors counting occurrences of each word, but cannot capture that 'cat' and 'dog' are semantically related (both animals).

B. TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF improves on BoW by weighting words based on their importance. It gives more weight to words that are frequent in a document but rare across the corpus.

- TF (Term Frequency): How often a word appears in a document
- IDF (Inverse Document Frequency): log(Total documents / Documents containing the term)
- TF-IDF = TF x IDF
- Common words like 'the', 'is', 'a' get low scores (high frequency everywhere)
- Distinctive words like 'transformer', 'attention' get higher scores
- Limitation: Still does not capture word meaning or context

C. Word Embeddings

Word embeddings represent words as dense, continuous vectors in a lower-dimensional space where semantically similar words are closer together.

- Word2Vec (Google, 2013): Learns embeddings using either Skip-gram or CBOW (Continuous Bag of Words)
- GloVe (Stanford, 2014): Global Vectors for Word Representation, based on word co-occurrence statistics
- Key property: Captures semantic relationships - e.g., King - Man + Woman = Queen
- Vectors typically have 100-300 dimensions (compared to vocabulary-sized vectors in BoW)
- Limitation: Each word has only one vector regardless of context ('bank' as river bank vs. financial bank)

Evolution: BoW (counts) -> TF-IDF (weighted counts) -> Word Embeddings (learned dense vectors)

2.5 Neural Sequence Models: RNN & LSTM

Why Sequence Models?

Language is inherently sequential - the meaning of a word often depends on what came before it. Traditional models like N-grams and Bag of Words cannot effectively model these sequential dependencies. Neural sequence models were designed to process data in order, maintaining a 'memory' of what they have seen.

Recurrent Neural Network (RNN)

An RNN is a neural network that processes sequences by maintaining a hidden state that is updated at each time step. The hidden state acts as the network's 'memory' of previously processed inputs.

- At each time step, the RNN takes the current input and the previous hidden state to produce a new hidden state
- Formula: h_t = f(W_h * h_{t-1} + W_x * x_t + b), where h_t is hidden state, x_t is input
- Can process variable-length sequences
- Shares parameters across all time steps (weight sharing)
- Applications: Language modelling, text generation, machine translation

The Vanishing Gradient Problem

RNNs suffer from the vanishing gradient problem during training. When training with backpropagation through time (BPTT), gradients can become exponentially small as they propagate back through many time steps. This means:

- The network struggles to learn long-range dependencies
- Information from early parts of a sequence gets 'forgotten'
- Example: In 'The students who studied AI, ML, DL and NLP yesterday finally submitted the assignment', an RNN may struggle to connect 'students' with 'submitted'
- Conversely, exploding gradients can cause training instability

LSTM (Long Short-Term Memory)

LSTM is a special type of RNN designed to overcome the vanishing gradient problem. Introduced by Hochreiter & Schmidhuber (1997), LSTMs use a gating mechanism to control information flow.

Cell State (Memory Highway)

A dedicated pathway that allows information to flow through the entire sequence with minimal modification. Think of it as a conveyor belt running through the network.

Forget Gate

Decides what information to discard from the cell state. Uses a sigmoid function that outputs values between 0 (forget completely) and 1 (keep completely).

Input Gate

Decides which new information to add to the cell state. Combines a sigmoid (what to update) with a tanh (candidate values) layer.

Output Gate

Decides what information from the cell state to output as the hidden state. Filters the cell state through a sigmoid and tanh.

RNN vs LSTM - Key Differences:

- RNN: Single hidden state, prone to vanishing gradients, limited memory
- LSTM: Cell state + hidden state + three gates, better at long-range dependencies
- LSTM can selectively remember and forget information
- LSTMs were state-of-the-art for many NLP tasks before Transformers

Encoder-Decoder Architecture

The Encoder-Decoder (Sequence-to-Sequence) architecture is designed for tasks where the input and output are both sequences, potentially of different lengths.

- Encoder: Reads the entire input sequence and compresses it into a fixed-length context vector
- Decoder: Takes the context vector and generates the output sequence, one token at a time
- Applications: Machine translation, text summarization, question answering
- Problem: The fixed-length context vector becomes a bottleneck for long sequences
- Solution: The Attention mechanism (see next section)

2.6 The Attention Mechanism

Attention is a mechanism that allows the model to focus on relevant parts of the input when producing each part of the output, rather than relying on a single fixed context vector.

Core Idea

Instead of compressing the entire input into one vector, attention lets the decoder 'look back' at all encoder hidden states and assign different weights (attention scores) to each. The decoder can focus on the most relevant parts of the input for each output step.

How Attention Works

- Step 1: The encoder produces a hidden state for each input token
- Step 2: For each decoder step, compute a score between the decoder's current state and each encoder state
- Step 3: Apply softmax to get attention weights (probabilities that sum to 1)
- Step 4: Compute a weighted sum of encoder states using these weights = context vector for this step
- Step 5: Use this context vector along with the decoder state to generate the output

Key Insight: Attention = 'Focus on what matters.' It replaced the information bottleneck of fixed-size context vectors.

The Evolution So Far

- N-grams: Looked at nearby words only
- RNNs: Remembered previous words sequentially
- LSTMs: Remembered important information selectively
- Attention: Learned what to focus on dynamically
- Next: Transformers - Attention is all you need!

2.7 Transformers

The Transformer architecture, introduced in the landmark paper 'Attention Is All You Need' (Vaswani et al., 2017), revolutionized NLP by replacing recurrence with self-attention. It is the foundation of virtually all modern language models.

Why Transformers Replaced Recurrent Models

- RNNs/LSTMs process tokens sequentially - slow, cannot be parallelized
- Transformers process all tokens simultaneously - massively parallelizable on GPUs
- Self-attention captures relationships between all pairs of tokens regardless of distance
- Better at learning long-range dependencies than RNNs or even LSTMs
- Enabled training on much larger datasets due to computational efficiency

Self-Attention

Self-attention allows each token in a sequence to attend to every other token, computing a representation that captures the relationships between all positions.

Query, Key, Value (Q, K, V)

Self-attention uses three learned projections of each token's embedding:

- Query (Q): 'What am I looking for?' - Represents the current token's search criteria
- Key (K): 'What do I contain?' - Represents what each token offers
- Value (V): 'What information do I provide?' - The actual content to aggregate
- Attention Score = softmax(Q * K^T / sqrt(d_k)) * V
- The division by sqrt(d_k) prevents the dot products from growing too large (scaled dot-product attention)

Attention Scores Computation

For each token, attention scores are computed as follows:
1. Compute dot product of Query with all Keys to get raw scores
2. Scale by dividing by sqrt(d_k) where d_k is the key dimension
3. Apply softmax to get attention weights (probabilities)
4. Multiply weights by Values to get the weighted representation
5. The result is a context-aware representation of each token

Multi-Head Attention

Instead of performing a single attention computation, the Transformer uses multiple attention 'heads' in parallel, each learning different types of relationships.

- Each head has its own Q, K, V projection matrices
- Different heads can capture different types of relationships (syntactic, semantic, positional)
- Outputs from all heads are concatenated and projected through a linear layer
- Typical models use 8 to 96 attention heads
- Analogy: Like having multiple 'experts' each looking at the data from a different perspective

Positional Encoding

Since Transformers process all tokens simultaneously (no inherent notion of order), positional encodings are added to give the model information about token positions.

- Uses sinusoidal functions of different frequencies: PE(pos, 2i) = sin(pos / 10000^(2i/d_model))
- Even dimensions use sine, odd dimensions use cosine
- Allows the model to learn relative positions
- Alternative: Learned positional embeddings (used in GPT)

The Transformer Block

A single Transformer block consists of:

- Multi-Head Self-Attention layer
- Add & Norm (residual connection + layer normalization)
- Feed-Forward Network (two linear layers with activation)
- Add & Norm (another residual connection + layer normalization)
- Modern LLMs stack many such blocks (GPT-3 has 96 layers, GPT-4 has even more)

Encoder vs Decoder

- Encoder (e.g., BERT): Bidirectional attention - each token attends to all others. Good for understanding tasks (classification, NER)
- Decoder (e.g., GPT): Causal/masked attention - each token can only attend to previous tokens. Good for generation tasks
- Encoder-Decoder (e.g., T5, BART): Encoder processes input, decoder generates output. Good for translation, summarization

RNN vs Transformer Comparison

- Processing: RNN = Sequential | Transformer = Parallel
- Long-range dependencies: RNN = Struggles | Transformer = Handles well via self-attention
- Training speed: RNN = Slow (sequential) | Transformer = Fast (parallelizable)
- Memory: RNN = Hidden state | Transformer = Attention over all positions
- Scalability: RNN = Limited | Transformer = Scales to billions of parameters

Chapter 3: Module 2 - GPT, LLMs & Applications

3.1 Large Language Models (LLMs)

From Language Model to LLM

A language model predicts the probability of the next token given previous tokens. A Large Language Model (LLM) is a language model distinguished by its massive scale - in parameters, training data, and compute.

Parameters

Parameters are the learned numerical values (weights and biases) inside a neural network that are adjusted during training. They encode the patterns the model has learned.

- GPT-2: 1.5 billion parameters
- GPT-3: 175 billion parameters
- GPT-4: Estimated 1+ trillion parameters (exact number not disclosed)
- LLaMA-2: 7B, 13B, 70B parameter variants
- More parameters generally = more capacity to learn complex patterns (but also more expensive)

Training Data

Training data is the information from which a model learns patterns. The quality, diversity, and scale of training data significantly impact model capabilities.

- LLMs are trained on massive internet-scale datasets (books, websites, code, academic papers)
- GPT-3 was trained on ~570 GB of filtered text data
- Data quality matters as much as quantity - garbage in, garbage out
- Training data has a cutoff date - the model does not know about events after training
- Bias in training data leads to bias in model outputs

Model Size vs Training Data

The relationship between model size and training data follows scaling laws. Research (Chinchilla scaling laws) suggests that model size and training data should scale proportionally for optimal performance. Undertrained large models can be outperformed by smaller models trained on more data.

Context vs Training Knowledge

Context is the information available to the model during a specific interaction (the prompt, conversation history). Training knowledge is everything the model learned during pre-training. These are fundamentally different:

- Training knowledge: Static, frozen after training, broad but may be outdated
- Context: Dynamic, provided at inference time, specific to the current task
- The model cannot update its training knowledge during inference
- Context window determines how much runtime information the model can use

Inference

Inference is the process of using a trained model to generate predictions or responses. Training teaches the model; inference uses what it learned.

- Autoregressive generation: The model generates one token at a time, feeding each output back as input
- Sampling strategies: Greedy (always pick highest probability), Top-k (sample from top k tokens), Top-p/Nucleus (sample from smallest set with cumulative probability >= p)
- Inference is computationally expensive for large models
- Techniques like quantization and distillation reduce inference cost

What Can LLMs Do?

- Text generation, summarization, and translation
- Question answering and information retrieval
- Code generation and debugging
- Reasoning and problem-solving (with limitations)
- Creative writing and brainstorming
- Few-shot and zero-shot task completion

Limitations of LLMs

- Hallucination: LLMs can generate plausible-sounding but factually incorrect information
- Knowledge cutoff: Cannot access information beyond their training data
- No true understanding: Pattern matching, not genuine comprehension
- Bias: Reflects biases present in training data
- Context window limits: Cannot process arbitrarily long inputs
- Lack of real-time information: Cannot browse the web or access live data (without tools)
- Reasoning limitations: May fail on complex logical or mathematical reasoning

Model Landscape

- OpenAI: GPT-3.5, GPT-4, GPT-4o, o1, o3
- Google: PaLM, Gemini, Gemma
- Meta: LLaMA, LLaMA-2, LLaMA-3
- Anthropic: Claude, Claude 2, Claude 3, Claude 3.5 Sonnet

- Mistral: Mistral 7B, Mixtral 8x7B
- Open-source models continue to close the gap with proprietary models

3.2 GPT: Architecture & Training

What is GPT?

GPT stands for Generative Pre-trained Transformer. It is a Transformer-based language model that is pre-trained on large amounts of data and generates text by predicting the next token.

- Generative: It generates new content (text, code, etc.)
- Pre-trained: It first learns general language patterns from massive datasets before being specialized
- Transformer: Built on the Transformer decoder architecture

GPT Architecture: Decoder-Only Transformer

GPT uses a decoder-only Transformer architecture designed for autoregressive text generation. Unlike the original Transformer which has both encoder and decoder, GPT only uses the decoder part with causal (masked) self-attention.

Causal / Masked Self-Attention

In GPT, each token can only attend to tokens that came before it (and itself). Future tokens are masked during training so the model cannot 'cheat' by looking at the answer. This is implemented using an attention mask that sets future positions to negative infinity before the softmax operation.

GPT History & Evolution

- GPT-1 (2018, OpenAI): 117M parameters. Demonstrated that unsupervised pre-training + supervised fine-tuning works
- GPT-2 (2019): 1.5B parameters. Showed emergent zero-shot abilities. Initially withheld due to misuse concerns
- GPT-3 (2020): 175B parameters. Breakthrough in few-shot learning. Powered early AI applications
- GPT-3.5 / ChatGPT (2022): Added instruction tuning and RLHF. Made AI accessible to the public
- GPT-4 (2023): Multimodal (text + images). Significantly improved reasoning and safety
- GPT-4o (2024): Optimized for speed and multimodal native capabilities

The important story is not the model names but the progression of architecture, scale, training methodology, and post-training alignment techniques.

How GPT is Trained

1. Pre-training

Pre-training is the large-scale process where the model learns language patterns by predicting the next token from massive datasets. The model reads billions of tokens from the internet and learns grammar, facts, reasoning patterns, and writing styles. Objective: Given tokens w_1, ..., w_{t-1}, predict w_t. This is unsupervised

(self-supervised) learning.

2. Fine-tuning

Fine-tuning takes a pre-trained model and trains it further for a particular behavior, domain, or task. It uses a smaller, curated dataset specific to the desired application. This adapts the general model into a specialized one.

3. Instruction Tuning

Instruction tuning teaches a language model to behave more like an assistant. The model is trained on datasets of (instruction, response) pairs so it learns to follow instructions rather than just completing text. This bridges the gap between 'text completion' and 'helpful assistant'.

4. RLHF (Reinforcement Learning from Human Feedback)

RLHF aligns model behavior with human preferences. Process: (1) Generate multiple responses to a prompt, (2) Human raters rank the responses, (3) Train a reward model on these rankings, (4) Use reinforcement learning (PPO) to optimize the language model against the reward model. This makes outputs more helpful, harmless, and honest.

Alignment

Alignment is about making model behavior match intended human goals, preferences, usefulness, and safety requirements. RLHF is the primary technique for alignment in modern LLMs.

GPT Variants & ChatGPT

GPT is a family of models. ChatGPT is a product/system built around GPT models. ChatGPT adds conversation management, safety filters, system prompts, and a user interface on top of the underlying GPT model.

How GPT Generates a Response

- 1. Tokenize the input prompt into tokens
- 2. Pass tokens through the Transformer decoder stack
- 3. At each position, compute causal self-attention (only attend to previous tokens)
- 4. The final layer produces a probability distribution over the vocabulary
- 5. Sample the next token based on temperature and sampling strategy
- 6. Append the new token to the sequence and repeat until a stop condition

3.3 Prompt Engineering

What is Prompt Engineering?

Prompt engineering is the practice of designing instructions and context that guide an AI model toward a desired output. It is the primary way users interact with and control LLMs.

The Three Pillars of a Prompt

1. Instruction

What do you want the model to do? The task description. Be specific and clear. Example: 'Summarize the following article in 3 bullet points.'

2. Context

What information does the model need to perform the task? Background information, data, or reference material. Example: Providing the article text to be summarized.

3. Constraints

What boundaries should the model follow? Output format, length, tone, what to include/exclude. Example: 'Keep each bullet point under 20 words. Use formal tone.'

Prompting Strategies

Zero-Shot Prompting

Asking the model to perform a task without providing any examples. Relies entirely on the model's pre-trained knowledge. Example: 'Translate the following to French: Hello, how are you?'

One-Shot Prompting

Providing one example before the actual task. Helps the model understand the expected format and style. Example: 'Translate English to French. English: Good morning -> French: Bonjour. English: Thank you -> French: ?'

Few-Shot Prompting

Providing multiple examples (typically 2-5) before the task. More examples help the model understand patterns more reliably. Especially useful for complex or ambiguous tasks.

Role Prompting

Giving the model a specific role, persona, or expertise to adopt while completing a task. Example: 'You are an expert data scientist. Explain neural networks to a business executive.' This shapes the tone, depth, vocabulary, and perspective of the response.

Prompt Engineering Framework

- Define the task clearly (What do you want?)

- Provide necessary context (What does the model need to know?)
- Set constraints (Format, length, tone, boundaries)
- Choose a strategy (Zero-shot, Few-shot, Role-based)
- Iterate and refine based on outputs

3.4 Advanced Prompt Engineering

Advanced prompt engineering moves beyond basic asking to designing the AI's behavior. The goal is reliable, consistent, high-quality outputs through systematic techniques.

Why Basic Prompts Fail

The core problem: the model has too much freedom. Without structure, the model may interpret instructions differently each time, produce inconsistent formats, or miss important requirements.

Delimiters

Delimiters are 'fences around different pieces of information.' They clearly separate instructions from data, preventing prompt injection and improving clarity.

- Triple backticks (```), triple quotes ("""), XML tags, or other clear separators
- Help the model distinguish between instructions and content to process
- Prevent accidental prompt injection from user input
- Example: 'Summarize the text delimited by triple backticks: ```...text...```'

Structured Prompting

Organizing prompts with clear sections, numbered steps, and labeled components makes them more reliable and easier to maintain.

XML Tags in Prompts

XML tags provide explicit boundaries between prompt components. Example: <instruction>...</instruction> <context>...</context> <output_format>...</output_format>

JSON Output Format

Requesting JSON output produces machine-readable, structured responses that can be directly parsed by downstream systems. Specify the exact JSON schema you expect.

Templates & Parameterization

Don't rewrite prompts for every task. Create templates with variables/placeholders that can be filled in programmatically. This ensures consistency across similar tasks.

Constraints

Constraints reduce ambiguity. Examples: word/character limits, required format, tone specifications, what to

include/exclude, language requirements.

Few-Shot Examples in Advanced Prompts

Show, don't just tell. Examples teach patterns more effectively than descriptions. Include input-output pairs that demonstrate the exact behavior you want.

Combining Techniques

Strong prompts combine multiple techniques: role + delimiters + structured format + examples + constraints. The best prompt isn't the longest - it's the one that removes the ambiguity that matters.

Prompt Chaining

Prompt chaining breaks complex tasks into a series of simpler, sequential prompts where the output of one becomes the input of the next.

- Step 1: Extract key information from a document
- Step 2: Use extracted information to generate a summary
- Step 3: Use the summary to create action items
- Each step is simpler and more reliable than trying to do everything in one prompt
- Allows error checking and human review between steps

Prompt Evaluation

Reliability does not come from better wording alone. Systematic evaluation is needed:

- Test prompts with diverse inputs to check consistency
- Define success criteria before evaluating
- Use automated evaluation where possible (exact match, semantic similarity, LLM-as-judge)
- Track prompt versions and their performance
- A/B test different prompt formulations

3.5 Image/Video Generation & Diffusion Models

What is Diffusion?

Diffusion models are a class of generative models that work by learning to reverse a gradual noising process. The core principle: learn to destroy (add noise), then learn to reconstruct (remove noise).

Forward Diffusion Process

Starting from a clean image, Gaussian noise is gradually added over many time steps until the image becomes pure random noise. This process is fixed and requires no learning.

Reverse Diffusion Process

A neural network is trained to reverse the noising process - given a noisy image, predict and remove the noise to recover the original image. At generation time, start from pure random noise and iteratively denoise to produce a clean image.

Text-to-Image Generation

Text-to-image models combine diffusion with text understanding to generate images from natural language descriptions.

- DALL-E (OpenAI): Pioneered text-to-image generation
- Stable Diffusion (Stability AI): Open-source diffusion model
- Midjourney: Known for artistic, high-quality outputs
- Process: Text prompt -> Text encoder -> Cross-attention with diffusion model -> Generated image
- The text encoder (often CLIP) converts the text into embeddings that guide the denoising process

Image-to-Image & Editing

Beyond generation from scratch, diffusion models enable image editing, style transfer, inpainting (filling in missing parts), outpainting (extending images), and image-to-image translation guided by text prompts.

Limitations

- Struggles with hands, fingers, and fine anatomical details
- Text rendering in images is often incorrect
- May generate biased or stereotypical imagery
- Computationally expensive (many denoising steps)
- Copyright and ethical concerns around training data
- Generative does not equal perfect - always review outputs

3.6 Multimodal AI

What is Multimodal AI?

Multimodal AI refers to AI systems that can understand, reason about, and generate across multiple types of data (modalities) - text, images, audio, video, and more. The key capabilities: Understand, Reason, and Generate across modalities.

How Multimodal Models Work

Different inputs (text, image, audio) are processed by specialized encoders and mapped into a shared representation space where the model can reason across modalities.

- Each modality has a specialized encoder (text encoder, vision encoder, audio encoder)
- Encoders map inputs into a shared embedding space
- A fusion mechanism combines representations from different modalities
- The model reasons over the combined representation
- Output can be in any supported modality

Applications of Multimodal AI

- Image + Text Understanding: Visual question answering, image captioning
- Video Understanding: Video summarization, action recognition
- Multimodal AI in Coding: Understanding diagrams/screenshots to generate code
- Document Understanding: Processing documents with text, tables, and images
- Accessibility: Describing images for visually impaired users

Key Insight: Multimodal AI is not just AI that can see, hear, and read. The real goal is AI that can combine those signals to understand, reason, and act.

Limitations of Multimodal AI

- Alignment between modalities is imperfect
- Computationally very expensive
- Hallucination can occur in any modality
- Training data biases are amplified across modalities
- Evaluation is more complex than single-modality models

3.7 Ethical Considerations in Generative AI

As Generative AI becomes more powerful and widespread, understanding its ethical implications is essential for responsible development and use.

Key Ethical Concerns

1. Bias and Fairness

Generative models learn from training data that reflects existing societal biases. This can lead to biased outputs in terms of gender, race, culture, and other dimensions. Mitigation: Diverse training data, bias auditing, fairness metrics, human review.

2. Misinformation and Deepfakes

Generative AI can create highly realistic fake content (text, images, audio, video) that can be used to spread misinformation, manipulate public opinion, or impersonate individuals. Mitigation: Watermarking, detection tools, digital provenance standards.

3. Hallucination

LLMs can generate plausible-sounding but factually incorrect information with high confidence. This is particularly dangerous in high-stakes domains like healthcare, law, and finance. Mitigation: RAG (Retrieval-Augmented Generation), human verification, confidence calibration.

4. Privacy Concerns

Models may inadvertently memorize and reproduce personal information from training data. Mitigation: Differential privacy, data filtering, access controls.

5. Intellectual Property

Questions about who owns AI-generated content, whether AI training on copyrighted material constitutes fair use, and how to attribute AI-assisted work. Ongoing legal debates worldwide.

6. Job Displacement

Automation of creative and knowledge work raises concerns about economic impact. Counter-argument: AI as augmentation rather than replacement; new job creation.

7. Environmental Impact

Training large models consumes enormous computational resources and energy. GPT-3 training estimated to have consumed ~1,287 MWh of energy. Mitigation: Efficient architectures, model distillation, green computing.

8. Responsible AI Principles

Transparency, accountability, fairness, privacy, safety, and human oversight are the core principles guiding responsible AI development. Organizations should implement AI governance frameworks and ethical review processes.

3.8 Generative AI Tools & Applications

Generative AI tools span multiple domains and industries. Here is an overview of tools and their applications in each category:

Storytelling

- ChatGPT / Claude / Gemini: Generate creative stories, plot outlines, character development
- Sudowrite: AI writing assistant specifically for fiction
- NovelAI: Specialized in narrative generation
- Use case: Writers use AI for brainstorming, overcoming writer's block, and drafting

Text-to-Image

- DALL-E 3 (OpenAI): High-quality image generation integrated with ChatGPT
- Midjourney: Artistic, highly detailed image generation via Discord
- Stable Diffusion: Open-source, customizable, runs locally
- Adobe Firefly: Commercially safe, trained on licensed content

Chatbots

- ChatGPT (OpenAI): Most widely used conversational AI
- Claude (Anthropic): Known for safety and long-context capabilities
- Gemini (Google): Multimodal capabilities integrated with Google services
- Applications: Customer service, education, healthcare triage, personal assistance

Text-to-Music

- Suno AI: Generate full songs with vocals from text descriptions
- Udio: High-quality AI music generation
- MusicLM (Google): Text-to-music research model
- AIVA: AI composer for background music and soundtracks

Language Translation

- Google Translate (with LLM integration): Real-time translation across 100+ languages
- DeepL: High-quality neural machine translation
- ChatGPT / Claude: Contextual translation with nuance understanding
- Meta's NLLB (No Language Left Behind): Open-source, 200+ languages

Travel Itinerary

- ChatGPT / Claude: Generate personalized travel plans based on preferences
- Wonderplan: AI-powered trip planning
- Roam Around: AI travel planner with destination suggestions
- Use case: Budget planning, restaurant recommendations, activity scheduling

Presentation Tools

- Gamma: AI-powered presentation creation from text prompts
- Tome: Generate entire presentations with narratives and visuals
- Beautiful.ai: AI-assisted slide design and formatting
- Microsoft Copilot in PowerPoint: AI integration in existing tools

Team Building

- AI-powered icebreaker generators and team activity planners
- Personality assessment tools enhanced by AI
- Virtual team-building facilitation using conversational AI
- Collaborative brainstorming tools (Miro AI, FigJam AI)

Education

- Khan Academy's Khanmigo: AI tutor for personalized learning
- Duolingo Max: AI-powered language learning with conversation practice
- GitHub Copilot: AI pair programming for coding education
- Quizlet AI: Automated flashcard and quiz generation
- NotebookLM (Google): AI research and study assistant

3.9 RAG (Retrieval-Augmented Generation)

What is RAG?

RAG (Retrieval-Augmented Generation) is an AI framework that improves LLM outputs by pulling fresh, external data before generating a response. It combines the power of information retrieval with the generative capabilities of LLMs.

Why RAG?

- Stops AI from guessing or making up facts (reduces hallucination)
- Lets AI access up-to-date or private information without expensive retraining
- Provides verifiable sources for generated answers
- Cost-effective alternative to fine-tuning for knowledge-intensive tasks

How RAG Works

- 1. Document Ingestion: External documents are split into chunks and converted into vector embeddings
- 2. Indexing: Embeddings are stored in a vector database for efficient similarity search
- 3. Query: When a user asks a question, the query is also converted to an embedding
- 4. Retrieval: The system finds the most relevant document chunks using similarity search
- 5. Augmentation: Retrieved context is added to the user's prompt
- 6. Generation: The LLM generates a response grounded in the retrieved information

Key Components

- Embedding Model: Converts text to dense vector representations (e.g., sentence-transformers)
- Vector Database: Stores and searches embeddings efficiently (e.g., FAISS, Pinecone, ChromaDB)
- Chunking Strategy: How documents are split (fixed-size, semantic, recursive)
- Retrieval Strategy: How relevant documents are found (similarity search, hybrid search)
- LLM: Generates the final response using retrieved context

Practical Project: Mini RAG Assistant - Chat with your documents using embeddings, vector search, and LLM generation.

3.10 MCP (Model Context Protocol)

What is MCP?

MCP (Model Context Protocol) is a lightweight protocol that connects AI applications to external tools, data, and APIs. It standardizes how AI models interact with the outside world.

The USB-C Analogy

Think of MCP as a USB-C port for AI. Just as USB-C provides a standardized connection between devices and peripherals, MCP provides a standardized connection between AI models and external tools/data sources.

How MCP Works

- Discover: AI client discovers available tools and resources exposed by MCP servers
- Request: The AI selects a relevant tool based on the user's prompt
- Execute: The MCP server interacts with external systems (APIs, databases, services)
- Return: Results are sent back to the AI, which incorporates them into its response

Why MCP Matters

- Standardizes integrations across AI applications and services
- Reduces custom connector development and maintenance
- Enables AI to move beyond generating responses to interacting with real-world tools and data
- Supported by major AI platforms (Anthropic, OpenAI, various IDEs)

Practical Project: Build your own MCP server exposing tools, then connect it to a compatible AI client.

Chapter 4: Key Formulas & Concepts Reference

Attention Formula

Attention(Q, K, V) = softmax(Q * K^T / sqrt(d_k)) * V

Where: Q = Query matrix, K = Key matrix, V = Value matrix, d_k = dimension of keys

TF-IDF Formula

TF(t,d) = (Number of times term t appears in document d) / (Total number of terms in d)

IDF(t) = log(Total number of documents / Number of documents containing term t)

TF-IDF(t,d) = TF(t,d) x IDF(t)

Softmax Function

softmax(z_i) = exp(z_i) / sum(exp(z_j)) for all j

Converts raw scores into probabilities that sum to 1.

RNN Update Rule

h_t = tanh(W_h * h_{t-1} + W_x * x_t + b)

Where: h_t = hidden state at time t, x_t = input at time t, W_h, W_x = weight matrices

Autoregressive Generation

P(x_1, x_2, ..., x_T) = Product of P(x_t | x_1, ..., x_{t-1}) for t=1 to T

Each token is generated conditioned on all previously generated tokens.

Positional Encoding

PE(pos, 2i) = sin(pos / 10000^(2i/d_model))

PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))

Key Architecture Comparison

RNN: Sequential processing, hidden state memory, vanishing gradient problem

LSTM: Gated RNN, cell state + hidden state, better long-range memory

Transformer: Parallel processing, self-attention, positional encoding, scalable

GPT: Decoder-only Transformer, causal attention, autoregressive generation

BERT: Encoder-only Transformer, bidirectional attention, masked language model

Chapter 5: Practice Questions

The following practice questions cover all major topics from both modules. Use them for self-assessment and exam preparation.

Module 1: Sample Questions

Short Answer / Explanation (5-8 marks each)

Q. Define Generative AI. Explain its scope and list any four applications of generative models. [7-8 marks]

Q. What are N-gram language models? Explain with examples of unigram, bigram, and trigram. State their limitations. [7-8 marks]

Q. Compare Bag of Words, TF-IDF, and Word Embeddings as word representation techniques. [7-8 marks]

Q. Explain the architecture of a Recurrent Neural Network (RNN). What is the vanishing gradient problem? [7-8 marks]

Q. How does LSTM solve the vanishing gradient problem? Explain the role of forget gate, input gate, and output gate. [7-8 marks]

Q. What is the Attention mechanism? Why was it introduced and how does it improve the encoder-decoder architecture? [7-8 marks]

Q. Explain the Transformer architecture. Describe self-attention, multi-head attention, and positional encoding. [7-8 marks]

Q. Compare RNN, LSTM, and Transformer architectures in terms of processing, memory, and scalability. [7-8 marks]

Q. What is the difference between encoder-only, decoder-only, and encoder-decoder Transformer architectures? Give examples. [7-8 marks]

Q. Explain how self-attention computes Query, Key, and Value. What is scaled dot-product attention? [7-8 marks]

Q. Discuss the ethical considerations and challenges associated with Generative AI. [7-8 marks]

Q. Describe the importance of Generative AI in at least four different domains. [7-8 marks]

Module 2: Sample Questions

Short Answer / Explanation (5-8 marks each)

Q. What does GPT stand for? Explain the architecture and working of GPT models. [7-8 marks]

Q. Describe the GPT training pipeline: pre-training, fine-tuning, instruction tuning, and RLHF. [7-8 marks]

Q. What is RLHF? How does it help in aligning AI model behavior with human preferences? [7-8 marks]

Q. Explain the concept of tokens, context window, and temperature in Generative AI. [7-8 marks]

Q. Compare different GPT variants. How has GPT evolved from GPT-1 to GPT-4? [7-8 marks]

Q. What is ChatGPT? How is it different from the underlying GPT model? [7-8 marks]

Q. Define prompt engineering. Explain zero-shot, one-shot, and few-shot prompting with examples. [7-8 marks]

Q. Explain advanced prompt engineering techniques: delimiters, structured prompting, and prompt chaining. [7-8 marks]

Q. What are diffusion models? Explain the forward and reverse diffusion process. [7-8 marks]

Q. What is Multimodal AI? How do multimodal models process different types of inputs? [7-8 marks]

Q. Discuss the ethical implications of generative models including bias, misinformation, and hallucination. [7-8 marks]

Q. Describe any five Generative AI tools and their real-world applications across different domains. [7-8 marks]

Q. What is RAG (Retrieval-Augmented Generation)? Explain how it reduces hallucination in LLMs. [7-8 marks]

Q. Explain the concept of MCP (Model Context Protocol) and how it enables AI to interact with external tools. [7-8 marks]

Q. Explain the role of LLMs in various industries. What are their key capabilities and limitations? [7-8 marks]

Chapter 6: References & Recommended Reading

Recommended Textbooks

- Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play - by David Foster
- Language Models for Natural Language Processing - by Rajendra Prasad Yadav, M. Shehbaz Khan, and Deepak Gupta
- Deep Learning - by Ian Goodfellow, Yoshua Bengio, and Aaron Courville

Reference Books

- Speech and Language Processing - by Daniel Jurafsky and James H. Martin

Key Papers

- Attention Is All You Need (Vaswani et al., 2017) - The original Transformer paper
- Improving Language Understanding by Generative Pre-Training (Radford et al., 2018) - GPT-1 paper
- Language Models are Few-Shot Learners (Brown et al., 2020) - GPT-3 paper
- Training Language Models to Follow Instructions with Human Feedback (Ouyang et al., 2022) - InstructGPT/RLHF
- Denoising Diffusion Probabilistic Models (Ho et al., 2020) - Foundational diffusion model paper
- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020) - RAG paper

Online Resources

- OpenAI Research: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
- TechTarget - Generative AI: https://www.techtarget.com/searchenterpriseai/definition/generative-AI
- UCL - Introduction to Generative AI: https://www.ucl.ac.uk/teaching-learning/generative-ai-hub/introduction-generative-ai
- TechTarget - Generative AI Examples: https://www.techtarget.com/searchenterpriseai/tip/Successful-generative-AI-examples-worth-noting

Hands-on Resources

- BertViz - Transformer Attention Visualization: https://github.com/Hrithik-Gavankar/demo1-bertviz
- minGPT - Minimal GPT implementation: https://github.com/karpathy/minGPT
- PyTorch Official Examples: Word-level language modelling and translation with Transformers